

Routing files (the special files)

Live demo: `/routing-files` · Code: `src/app/routing-files/`

What it is

Inside any route folder, certain **filenames are reserved** by Next.js. Each one plays a specific role in building a route segment. This one folder contains almost all of them so you can trigger each and see which file renders.

Files in this folder

File	Role	How to see it
<code>layout.tsx</code>	Shared shell that persists across navigation	The top box — its clock stays fixed
<code>template.tsx</code>	Shared shell that re-mounts every navigation	Second box — its clock changes
<code>page.tsx</code>	The route's unique UI (makes <code>/routing-files</code> public)	This index page
<code>loading.tsx</code>	Suspense fallback while a segment streams	Click Slow page
<code>error.tsx</code>	Error boundary (Client Component)	Click Crash → trigger error
<code>not-found.tsx</code>	UI for <code>notFound()</code> / unmatched routes	Click Missing
<code>api/route.ts</code>	An API endpoint (no UI)	Click Call the API

The root also has `global-error.tsx` and `not-found.tsx` (app-wide versions).

Key rules

- **layout vs template** : both wrap children. A layout is created once and preserves state across navigations within it; a template makes a *new* instance on every navigation (state resets, effects re-run). Use `template` for per-navigation effects (e.g. logging a pageview, enter animations).
- **error.tsx must be a Client Component** ("use client"). It receives `{ error, reset }`. It catches render errors **below** it — but **not** errors in its own segment's `layout`. For those, use `global-error.tsx` at the root.
- **Error boundaries catch render errors, not event-handler errors.** That's why the Crash demo flips state to throw during render.
- **page vs route** : a folder can expose UI (`page`) **or** an API (`route`), never both.
- `notFound()` throws a special control-flow signal that renders the nearest `not-found.tsx` ; code after it never runs.

Component hierarchy

These files nest in a fixed order: `layout` → `template` → `error` → `loading` → `not-found` → `page` . See the `/component-hierarchy` demo.

Interview Q&A

- **Q: Difference between `layout.tsx` and `template.tsx` ?** A layout persists and keeps state across navigation; a template re-mounts on every navigation, resetting state and re-running effects.
- **Q: Why must `error.tsx` be a Client Component?** Error boundaries rely on React class-component lifecycle `reset`

interactivity, which only runs on the client.

- **Q: What catches an error thrown in the root layout?** `global-error.tsx` (it replaces the root layout, so it renders its own `<html> / <body>`). A normal `error.tsx` cannot.
- **Q: Can a route have both a page and an API?** No — `page` and `route` are mutually exclusive in the same folder.